

A deep reinforcement learning based multiple meta-heuristic methods approach for resource constrained multi-project scheduling problem

Ziting Han

School of Automation
Southeast University
Nanjing, China
2287502039@qq.com

Yanlan Yang*

School of Automation
Southeast University
Nanjing, China

* Corresponding author: yyl@seu.edu.cn

Hua Ye

School of Automation
Southeast University
Nanjing, China

Abstract—In order to solve resource-constrained multi-project scheduling problem (RCMPSP) more efficiently, this paper proposes a deep reinforcement learning algorithm based multiple meta-heuristic methods that combines the advantages of discrete cuckoo search (DCS) and particle swarm optimization (PSO). In the process of population evolution, Deep reinforcement learning was used to select the most suitable meta-heuristic algorithm (DCS and PSO) according to the diversity and quality of the population, and the reward was designed according to the evolution effect, so as to guide the algorithm to update the solutions more effectively and find the optimal solution quickly. In addition, the key steps of the CS algorithm, Levy flight and random walk, are redefined in this paper. The task movement, reverse mutation, task list recombination and adaptive and repairable swap mutation are used to make it suitable for solving discrete RCMPSP problems and improve the convergence speed of DCS algorithm. Experimental results on the latest data set (MPLIB) demonstrate the effectiveness of the proposed algorithm.

Keywords: Resource-constrained multi-project scheduling problem; Deep reinforcement learning; Multiple meta-heuristic methods; Discrete cuckoo search; Particle swarm optimization

I. INTRODUCTION

The Resource-constrained multi-project Scheduling Problem (RCMPSP) is a typical NP-hard Problem. Compared with RCPSP, RCMPSP not only needs to consider the constraints of each task in a project, but also needs to consider the competition and sharing among projects, so resource allocation is more complex and the solution is much more difficult.

Meta-heuristic algorithm is widely used in solving RCMPSP with its wide application and high search efficiency [1]. Wang et al. [2] proposed a two-stage decomposition algorithm based on ant colony algorithm, which dealt with the timing constraints and resource constraints of the project successively. Chen et al. [3] constructed a normalized attribute set and coding method suitable for RCMPSP by analyzing the existing priority rules,

and evolved the best priority rules by using the improved Hyper-heuristic genetic algorithm. Wang et al. [4] adopted the adaptive particle swarm optimization algorithm with dynamically variable inertia weight to solve the RCMPSP problem aimed at minimizing the weighted duration.

The above meta-heuristic algorithms have their advantages and disadvantages in the performance and convergence of exploring the optimal solution. Cuckoo search (CS) algorithm can often achieve more ideal results in solving scheduling problems due to its good global performance and strong optimization ability [5], but has slow convergence speed for the lack of information communication among individuals. Particle swarm search (PSO) algorithm completes population evolution through information sharing and cooperation among individuals within a population, so its convergence speed is fast [6], but global search ability is poor.

Reinforcement learning has the characteristic of self-decision, which can be adjusted according to the state [7]. Therefore, in order to make use of the advantages of the two algorithms, this paper applies the reinforcement learning-Deep Q Network (DQN) algorithm to solve the RCMPSP problems, aiming at using its learning mechanism to select the appropriate evolutionary algorithm according to the current state of the population in the process of population evolution, so as to quickly find the global optimal solution. In this paper, the scheduling problem is transformed into a decision process of updating a series of solutions by defining state space, action space and reward function, and the RCMPSP is studied using DQN algorithm. In addition, the key steps of cuckoo algorithm, levy flight and random walk, were redefined in this paper, and the task movement, reverse mutation, task list recombination and adaptive and repairable swap mutation are used to make it suitable for solving discrete RCMPSP problems and improve the convergence speed of DCS.

II. RCMPSP MODEL

In this paper, the objective function is to minimize the makespan of multiple projects, and the RCMPSP model is established as follows:

Objective:

$$\min \max \{st_{ij}\} \quad (1)$$

Subject to:

$$st_{ij} - st_{ih} \geq d_{ih}, \forall h \in P_{ij}, \forall i, j \quad (2)$$

$$d_{i0}, d_{i(z+1)} = 0, r_{i0k}, r_{i(z+1)k} = 0, \forall i, k \quad (3)$$

$$\sum_{st_{ij} \in A_t} r_{ijk} \leq R_k, \forall k, t \quad (4)$$

$$t_{ij} \geq 0, d_{ij} \geq 0, r_{ijk} \geq 0 \quad (5)$$

Equation (1) is the objective function. (2) represents that tasks can only be started after all precursors are scheduled. (3) is virtual task constraint. (4) represents the total demand for resource k by tasks performed at time t cannot exceed its supply. (5) indicates that the task start time, task duration and resource are non-negative.

III. PROPOSED ALGORITHM

Meta-heuristic algorithm solving RCMPSP problem can be regarded as the evolution process of population. Different meta-heuristic algorithms, such as DCS and PSO, have different evolution modes, and their evolution effects are also different at different stages. An ideal meta-heuristic algorithm can adapt to different evolutionary stages of a population and select the appropriate evolutionary mode, so it is a multi-stage evolutionary decision problem. Reinforcement learning can obtain the feedback of the current environment and flexibly select the optimal decision, which is very suitable for solving the evolutionary decision problem. Therefore, in order to make use of the evolutionary characteristics of DCS and PSO at the same time, reinforcement learning algorithm is used to select the appropriate meta-heuristic algorithm according to the current state of the population in the process of population evolution.

A. State Representation

In this paper, population diversity and population quality were used as the state of population in reinforcement learning, as shown in (6) and (9).

$$div^t = \sqrt{\frac{1}{PS} \sum_{i=1}^{PS} (ms_i^t - \overline{ms}^t)^2} \quad (6)$$

$$\overline{ms}^t = \frac{1}{PS} \sum_{i=1}^{PS} ms_i^t \quad (7)$$

PS is the size of the population and $\overline{ms}^t$ is the average makespan of solutions in the t generation.

$$qual^t = \frac{ms_{best}^t}{NoTask} \quad (8)$$

$$qual_{norm}^t = \frac{qual^t - dt_{min}}{dt_{min}} \quad (9)$$

$$dt_{min} = \min_{\substack{i \in [1, n] \\ j \in [1, z]}} d_{ij} \quad (10)$$

ms_{best}^t is the makespan of the optimal solution in the t generation, $NoTask$ is the total number of tasks in all projects, and dt_{min} is the minimum duration of all tasks.

B. Actions for RL

In this paper, each meta-heuristic algorithm, the improved discrete cuckoo search algorithm and the hybrid particle swarm optimization is considered as an action of reinforcement learning.

1) Improved Discrete Cuckoo Search

a) Improved Levy Flight

In this paper, the Levy flight step obtained by Mantegna algorithm [8] was discretized and divided into small step and large step. The small step movement was defined as the movement of task position, the large jump was defined as the inversion mutation and the reorganization of task list, and the step size after discretization was the number of corresponding operations. Details of the scheme are as follows:

When levy's flight step is in $[0, 1]$, k_1 amounts of small steps are performed, k_2 amounts of small steps are performed in $[1, 2]$, and k_3 long jumps are performed in $[2, 6]$.

$$k_1 = \left\lceil Levy * \max_moves * \frac{2}{3} \right\rceil, Levy \in [0, 1] \quad (11)$$

$$k_2 = \left\lceil (Levy + 1) * \max_moves * \frac{1}{3} \right\rceil, Levy \in (1, 2] \quad (12)$$

$$k_3 = \left\lceil \frac{\lfloor Levy \rfloor}{2} \right\rceil, Levy \in (2, 6] \quad (13)$$

$\max_moves$ is the maximum number of possible moves, $\lceil x \rceil$ is the upward integer function, $\lfloor x \rfloor$ is the downward integer function.

The small step movement is designed as the position movement of the task in the event.

A set of tasks with the same start time is called an event [9] [10]. When moving tasks in the event, the number of moves to be performed is firstly calculated according to (11) and (12), that is, the number of tasks to be moved. Then a certain number of events are randomly selected to relocate the tasks in them. The relocation principle is to find the precursor with the latest end time and the successor with the earliest start time of the selected task, and insert it at random somewhere between the two tasks.

Inversion mutation and task list reorganization are selected equally when Levy flight performs a large jump.

Recombination of task lists increases the ability of the population to learn from other nests and breaks the limitation of slow convergence caused by the lack of information exchange between individuals in the traditional algorithm. Specifically, k_3 individuals are selected from the population by sorting selection and roulette and $2k_3$ new individuals are generated by a single point crossover operation with the current individuals. The new individual with the highest fitness is selected as the result of large jump.

b) Improved random walk

Adaptive adjustment is used in this paper to adjust the probability of eliminating nests according to their fitness value (14), and the resulting nests are regenerated by random walk. Furthermore, if the adaptation of the new nest is too low, it means that the random walk fails and needs to be redone. This paper defines the random walk as a swap mutation of a task list. Since the task list representing the bird nest is feasible and limited by predecessor constraints, traditional swap mutations may destroy this relationship. As seen in Fig. 1, the swap mutation has caused task 1.4 to be executed before its predecessor task 1.2. Therefore, the task list needs to be repaired to correct the tasks which fail to meet the constraints after their predecessor tasks.

$$P_a(c_i) = 1 - e^{\frac{f(c_i)}{f_{max}} - 1.1} \quad (14)$$

where $f(c_i)$ is the fitness of the nest c_i , and f_{max} is the current population's maximum fitness.

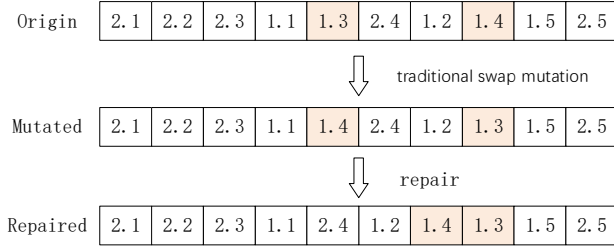


Figure 1. Swap mutation based on repair.

2) Hybrid particle swarm optimization

In this paper, the hybrid algorithm of DE and PSO proposed by Liu [11] is applied to solve RCMPSP. The algorithm uses priority-based encoding and SSG-based decoding to represent the particles, and carries out differential evolution operation for the global optimal particles, which not only ensures the population diversity but also slows down the speed of falling into the local optimal.

C. Reward Function

Since the ultimate goal of the algorithm is to quickly find the global optimal solution, that is, the optimal task scheduling scheme, we use the improvement rate of the solution to represent the reward function. The improvement rate of the solution consists of the probability of improved solutions and the number of breakthrough solutions whose fitness is higher than the highest fitness of the original population.

$$R(s, a) = \frac{1}{PS} \sum_{i=1}^{PS} NoImp_{i,a} + \sum_{i=1}^{PS} NoBonus_{i,a} \quad (15)$$

$$NoImp_{i,a} = \begin{cases} 1, & fit(c_i, a) > fit(c_i) \\ 0, & fit(c_i, a) \leq fit(c_i) \end{cases} \quad (16)$$

$$NoBonus_{i,a} = \begin{cases} 1, & fit(c_i, a) > fit_{max} \\ 0, & fit(c_i, a) \leq fit_{max} \end{cases} \quad (17)$$

$NoImp_{i,a}$ is the number of improved solutions after taking the action a , and $NoBonus_{i,a}$ is the number of breakthrough solutions. $fit(c_i, a)$ is the fitness of c_i after taking action the a , and fit_{max} is the maximum fitness in the original population.

D. DQN

Since the problem studied here is characterized by complex and continuous state space, DQN algorithm is used to solve it [12] [13], and Multilayer Perceptron (MLP) is used as the valuation network.

E. Algorithm framework

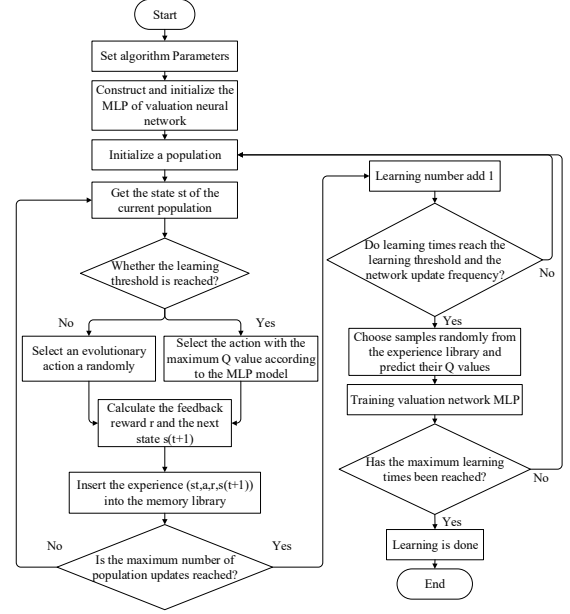


Figure 2. The framework of the proposed DCSPSOwRL Algorithm.

IV. EXPERIMENTAL RESULTS AND COMPARISONS

To evaluate the effectiveness of the proposed algorithm (DCSPSOwRL), experiments were carried out on MPLIB[14], the newly published dataset used to solve the RCMPSP problem. The algorithm parameters are provided in Table 1, and DCSPSOwRL was compared with its constituent algorithms DCS and PSO as well as the existing algorithm GA.

Fig. 3 illustrates the solving results of above algorithms for the same parameters on the test instance m1_S1_1. This shows that the PSO algorithm has the fastest convergence rate, but it quickly falls into the local optimal solution. The improved DCS algorithm has outstanding optimization performance and is superior to GA with regard to convergence speed and solution effect. And the DCSPSOwRL not only learns the convergence ability of PSO, but also the optimization ability of improved DCS algorithm. It has the best effect on finding the optimal solution quickly out of all comparison algorithms.

Moreover, DCSPSOwRL and its variant DCSPSOwR which chooses an evolutionary algorithm randomly in each iteration are compared in this paper with regard to the effectiveness of using RL to select the optimal algorithm in the optimization process. As shown in Fig. 4, DCSPSOwRL has a more accurate selection of evolutionary algorithms and a more robust overall search capability compared with DCSPSOwR.

The experimental results reveal that the DCSPSOwRL algorithm proposed in this paper combines the benefits of DCS

and PSO, uses RL as an adaptive selection mechanism in the evolution process and performs well in terms of searchability and convergence.

TABLE I. ALGORITHM PARAMETERS

Name	Value	Name	Value
<i>popSize</i>	60	<i>max_fes_num</i>	100
<i>Niter</i>	150	α	0.6
<i>start_num</i>	40	γ	0.75
<i>update_num</i>	10	<i>max_moves</i>	6

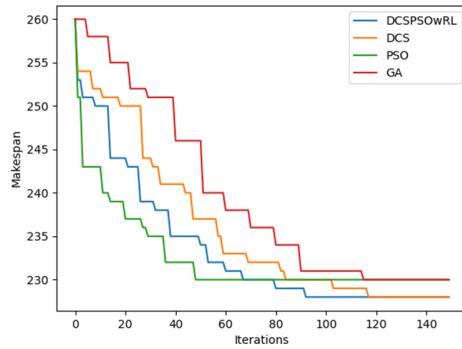


Figure 3. A comparison of DCSPSOwRL, Improved DCS, PSO and GA
Example of a figure caption.

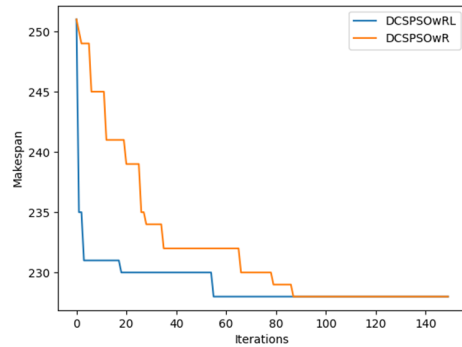


Figure 4. Performance comparison between DCSPSOwRL and DCSPSOwR.

V. CONCLUSION

This paper proposes a deep reinforcement learning algorithm that combines multiple meta-heuristics to solve RCMPSP problems.

RL technology is used to select the best meta-heuristic algorithm (DCS and PSO) based on the diversity and quality of the solution, and reward value is designed according to the evolution effect, so as to achieve effects of strong optimization ability, fast convergence speed and wide application range. In the process of evolution, RL is used to select the most suitable meta-heuristic algorithm (DCS and PSO) based on the diversity and quality of the solutions, and reward value is designed

according to the evolution effect, so as to achieve effects of strong optimization ability, fast convergence rate and wide application range. In addition, this paper redefines the key steps of the traditional CS algorithm: Levy flight and random walk making them suitable for solving discrete RCMPSP problems. At the same time, the task list recombination technology is used to improve the convergence speed of DCS algorithm.

By comparing the performance of the proposed algorithm DCSPSOwRL with its different components (DCS or PSO) and the existing algorithm GA on standard datasets, it can be seen that the DCSPSOwRL can obtain better results quickly in solving RCMPSP problems, and the improved DCS algorithm has faster convergence and stronger search ability compared with the traditional algorithm. In addition, the effectiveness of reinforcement learning is verified by comparison with the scheme of random evolutionary selection.

REFERENCES

- [1] Gonçalves J F, Mendes J J M, and Resende M G C, "A genetic algorithm for the resource constrained multi-project scheduling problem," *European journal of operational research*, vol. 189, pp. 1171-1190, Mar 2008.
- [2] Wang J Q, Zhang S F, Chen J, Zhang Y F, and Sun S D, "Decomposition algorithm for resource-constrained multi-project scheduling problem," *Computer Integrated Manufacturing Systems*, vol. 19, pp. 83-96, Jan 2013.
- [3] Chen H J, Ding G F, Zhang J, and Yan K Y, "Improved Genetic Programming Algorithm for Resource Constrained Multi-project Scheduling Problem," *China Mechanical Engineering*, vol. 32, pp. 1213, Oct 2021.
- [4] Wang H X, Wang Z H, Wen G F and Li H X, "Resource constrained multi-project scheduling based on adaptive particle swarm optimization algorithm," *Journal of Industrial Engineering/Engineering Management*, vol. 31, pp. 220-225, April 2017.
- [5] Ouaraab A, Ahiod B, and Yang X S, "Discrete cuckoo search algorithm for the travelling salesman problem," *Neural Computing and Applications*, vol. 24, pp. 1659-1669, July 2014.
- [6] Linyi D, Yan L. "A particle swarm optimization for resource-constrained multi-project scheduling problem," in *Conf. Computational Intelligence and Security, CIS, IEEE, 2007*, pp.1010-1014.
- [7] Vengerov D, "A reinforcement learning framework for utility-based scheduling in resource-constrained systems," *Future Generation Computer Systems*, vol. 25, pp. 728-736, July 2009.
- [8] Mantegna R N, "Fast, accurate algorithm for numerical simulation of Levy stable stochastic processes," *Physical Review E*, vol. 49, pp. 4677, May 1994.
- [9] Paraskevopoulos D C, Tarantilis C D, and Ioannou G, "Solving project scheduling problems with resource constraints via an event list-based evolutionary algorithm," *Expert Systems with Applications*, vol. 39, pp. 3983-3994, April 2012.
- [10] Bibiks K, Hu Y F, Li J P, Pillai P and Smith A, "Improved discrete cuckoo search for the resource-constrained project scheduling problem," *Applied Soft Computing*, vol. 69, pp. 493-503, 2018.
- [11] Liu J P, "Hybrid Particle Swarm Optimization Algorithm Based On Chaos and Differential Evolution," *Computer Simulation*, vol. 29, pp. 208-212, Feb 2012.
- [12] LeCun Y, Bengio Y, and Hinton G, "Deep learning," *nature*, vol. 521, pp. 436-444, 2015.
- [13] Li B S, Ye C M, "Job Shop Scheduling Problem Based on Deep Reinforcement Learning," *Computer Engineering and Applications*, vol. 23, pp. 248-254, 2021.
- [14] Van Eynde, R., Vanhoucke, M., "Resource-constrained multi-project scheduling: benchmark datasets and decoupled scheduling," *J Sched*, vol. 23, pp. 301-325, 2020.